9 Appendix

9.1 LLM instructions

Here are the instruction templates. Under the attribute recommendation setting, the attributes for validation and testing are excluded from the schema and hidden from the LLM.

Extraction Instruction (EI)

Your task is to analyze the given text, which may be about incidents, weather, or conversation, and extract the information according to the specified JSON structure below.
Provide the extracted information in the JSON structure {schema in json}.
Explanations about the JSON structure:
- Output only the JSON data.
- For each entity, there are multiple attributes.
- For each attribute, its value is a list of exact substrings of the input text. You should consider every attribute listed in the above structure. If an attribute is not present or cannot be determined, do not include the attribute in the output.

{dataset description}
Here are some examples: {examples}
Here is the input text: {text}
Here are some hints:
The above text is most likely to contain the attributes: $\{H^p\}$.
The above text is most likely to contain the following values: $\{V^p\}$.

The schema of each dataset is in Table 6. The dataset descriptions are:

Incidents - There can be three types of entities mentioned in the texts: Accident, Victim, and Suspect. There is at most one Accident in the text. There can be multiple Victims or Suspects in the text.

Weather - There is one type of entity mentioned in the texts: Weather. There can be multiple Weather in the text.

Conversation - Each input text is a multi-turn dialogue, involving at most 5 entities: Attraction, Hotel, Restaurant, Taxi, and Train. You should only extract the final agreed-upon state after each turn, covering all confirmed domains and their relevant attributes.

Discovery Instruction (DI)

Your task is to analyze the given text and discover new information mentioned in the text.
Provide the discovered information in this JSON structure : *{*

```
"Accident": {
    "0":{
        "⟨ new attribute 1⟩": [],
        "⟨new attribute 2⟩": []
    }
},
"Victim": {
    "0":{
        "⟨new attribute 1⟩": [],
```

"⟨new attribute 2⟩": []
},
"1":{...},
...
},
"Suspect": {
"0":{
"⟨new attribute 1⟩": [],
"⟨new attribute 2⟩": []
},
"1":{...},
...
},
}
Explanations about the JSON structure:
- Output only the JSON data.
- For each entity, there may be multiple attributes.
- For each attribute, its value is a list of exact substrings of the input text.
Here are some examples: {examples}
Explanations about the examples:
- Given an input text and the known information, you are encouraged to discover new information for each entity.
- The new information is qualified as a new attribute if and only if (1) it is relevant and important to the domain, (2) it is unique and not overlapped with the known information, and (3) it has a proper granularity level compared with the known attributes, which means it is not too high-level or low-level concepts.
- The new attribute must have a meaningful name and actual values from the text.
- Output qualified new attributes as many as possible.
Here is the input text: {text}
Here is the known information: {extracted table}

For other datasets, the *output formatting* are altered according to their schema.

Schema Prefix (SP)

Here is a schema designed to document and manage detailed information and facts about {dataset description}. For each entity, there are multiple attributes. The attributes are unique and in consistent styles. Here are the attributes in this schema:

The dataset descriptions are:

Incidents - criminal incidents or accidents, particularly those involving violence, such as shootings or other crimes. The attributes are organized into three main entities: the accident itself, the victims, and the suspects.

Weather - weather conditions from news reports, particularly those about location, time, temperature, wind, cloud, rain, and snow. The attributes are organized into one main entity: the Weather itself

Conversation - multi-turn dialogues, particularly those booking a service. The attributes are organized into five main entities: Attraction, Hotel, Restaurant, Taxi, and Train.

Judge Instruction (JI) in Hybrid Integration Strategy

Your task is to analyze the given schema and judge whether the given concepts are semantically similar enough to be merged as one attribute.
Here is the schema: {schema}
Here are the concepts to be judged: {attribute proposals}

Explanation about the task:
- The schema includes several unique attributes, which show the semantic granularity of attributes.
- If the above concepts (1) are semantically equivalent, or (2) one is semantically contained by the others, or (3) they represent a finer granularity compared with the existing attributes, they need to be merged into one single attribute. Then, output "One".
- Else, they should be at least two unique attributes, output "Two".
- Output only "Two" or "One".

9.2 Vendi Score

Vendi Score is the effective rank of the normalized similarity matrix,

$$vds(q) = \exp(-\mathrm{tr}(\frac{\mathcal{K}_{q\times q}}{|q|}\log\frac{\mathcal{K}_{q\times q}}{|q|})), \tag{23}$$

where $\mathcal{K}_{q\times q}$ is the similarity matrix of size $|q|\times|q|$ and $\mathrm{tr}(\cdot)$ means trace. Vendi Score is maximized as $|q|$ if $\mathcal{K}(i,j) = 0, \forall i,j \in q, i \neq j$, meaning every two elements are totally different, and it is minimized as 1 if $\mathcal{K}_{q\times q}$ contains all 1s. Since $q \subseteq Z \cup S = B$. We could precompute and cache $\mathcal{K}_{B\times B}$ and acquire submatix $\mathcal{K}_{q\times q}$ from it. Given $\mathcal{K}_{q\times q}$, the computational complexity of is in $O(|q|^3)$ [15]. In our experiment, we set $\mathcal{K}(a,b) = (\mathrm{chrf}\beta(a,b)+\mathrm{chrf}\beta(b,a))/2$, to reflect the pattern similarity as in Equation 9.

9.3 Supplementary Discussion for Hybrid Integration Strategy

9.3.1 Pruned bottom-up search for low-diversity set. As established by Equation 15, if a set is in P^*, then all of its subsets must also be in P^*. In other words, the monotonicity of $div(\cdot)$ means $div(p \cup \{z\}) \geq div(p), \forall z \notin p$. Leveraging this property, our algorithm computes P^* in ascending order of their size, enabling efficient pruning during the process, as in Algorithm 2.

Specifically, in each iteration of the outer loop, the algorithm examines all qualified sets of size len + 1. A dictionary Br maps each qualified set p of size len to a set of candidate attributes $\mathrm{Br}[p] \subseteq Z \cup S$ that may be added to p to form a larger qualified set. Within the inner loop, the algorithm considers a candidate extension $p' = p \cup \{z'\}$ for some $z' \in \mathrm{Br}[p]$. It first performs a fast dictionary lookup to verify that replacing any element $z \in p$ with z' yields a known qualified set. If this check succeeds, indicating that all proper subsets of p' are already known to be qualified, the algorithm computes the Vendi Score $vds(p')$ to decide the final answer.

9.3.2 Complexity of diversity checking. Let $N = |Z \cup S|$ be the total number of candidate attributes, and let $L = \max_{p\in P^*}|p|$ denote the size of the largest low-diversity set returned by the algorithm. For $i \in \{1,\ldots,L\}$, define $M_i = |\{p \in P^* : |p| = i\}|$, with $M_1 = N$. Algorithm 2 constructs P^* by attempting to extend each qualified set of size i with at most $N - i$ candidate attributes. The total

Algorithm 2 Pruned Bottom-up Search for P^*

Require: attribute proposals Z, known attributes S, threshold δ
Ensure: P^*

$P^* = \emptyset$, Br$\leftarrow$ {}, len$\leftarrow$ 1 ▷ Initialize dictionary.
for $p \in Z \cup S$ **do** Br$[p] = Z \cup S \setminus \{p\}$ ▷ Start by len=1.
end for
while |Br|$> 0 \wedge$len$<$ $|Z| + 1$ **do** ▷ Outer loop.
    nextBr$\leftarrow$ {}
    for $p \in$ Br, $z' \in$ Br$[e]$ **do** ▷ Inner loop.
        valid $\leftarrow$ *True*, activeBr $\leftarrow$ Br$[p]$
        for $z \in p$ **do**: ▷ Check all subset of length $|p|$
            $p' = p \setminus \{z\} \cup \{z'\}$ ▷ Replace one element, $|p'| = |e|$
            if $p' \in$ Br **then**
                activeBr $\leftarrow$ activeBr $\cap$ Br$[p']$ ▷ Prune.
            else
                valid$\leftarrow$ *False*
                Break
            end if
        end for
        if valid = *True* **then** ▷ All proper subset of p is in P^*.
            $p' = p \cup \{z'\}$
            if $vds(p') < 1 + \delta$ **then**
                $P^* \leftarrow P^* \cup \{p'\}$
                nextBr$[p']$ $\leftarrow$ activeBr
            end if
        end if
    end for
    len$\leftarrow$len+1, Br$\leftarrow$ nextBr ▷ Search for the larger size
end while

number of examined sets is therefore bounded by

$$|P'| \le \sum_{i=1}^{L} (N-i)M_i = N \sum_{i=1}^{L} M_i - \sum_{i=1}^{L} iM_i = N|P^*| - \sum_{p \in P^*} |p|.$$

Obviously, the number of evaluated sets scales linearly with the output size $|P^*|$, i.e., $|P'| = O(N|P^*|)$. Each candidate extension incurs $O(i)$ dictionary lookups, and a small fraction proceed to a Vendi Score computation, yielding an overall time complexity of $O\big(\sum_{i=1}^{L}(N-i)M_i \cdot (i + C_{vds})\big)$.

In practice, the pruning step within the inner loop further restricts the branch set Br$[p]$: the number of viable extensions for a set p never exceeds that of any of its subsets, and Br$[p]$ shrinks as p is extended. Hence, the factor $(N-i)$ can be tightened. Empirically, most candidate attributes are semantically distinct and do not form duplicate pairs with any other proposal; only a small fraction participate in true redundancies. Consequently, the factor is far smaller than N, and L remains modest (as detailed in the experimental section around Table 3).

9.3.3 Complexity of contextualized analysis. Suppose there are N' duplicates (i.e., elements that can potentially be eliminated by a *yes* response), and $|B| - N'$ non-duplicate elements. Then there must exist sets $q_1, q_2, \ldots, q_{|B|-N'}$ in P^* that together cover all elements, because every duplicate

must co-occur with at least one non-duplicate in some set belonging to P^*. Consequently, for the set cover instance defined by the collection P^* and the ground set B, the optimal solution size is strictly less than $|B| - N'$. A greedy set cover algorithm achieves an approximation ratio of $(\ln |B| + 1)$ [12]. Therefore, the number of LLM calls incurred by Q is at most $(\ln |B| + 1)(|B| - N')$. Line 10 further eliminates low-diversity sets that contain only a single element, which do not submit for LLM judgment.

Given the same B and $|P^*|$, in the worst case, the algorithm examines every set in P^* and receives *no* for all queries, incurring $|P^*|$ LLM calls without performing any deduplication. Such a situation is rare because each set in P^* is already known to be of very low diversity, meaning it plausibly contains duplicates. The analysis above indicates that a greater number of duplicates allows an outer iteration to complete more quickly, which submits each element at least once for duplication. We prioritize preserving the breadth of the search because the ultimate effect of deduplication hinges on whether each duplicate element is eliminated. The greedy set cover actually processes larger sets earlier, which helps the algorithm eliminate confirmed duplicates with few queries and shrinks the search space for subsequent iterations.

9.3.4 Discussion of Hybrid Integration Strategy. Resolving duplicate attribute proposals is essential, but this step lacks ground truth and thus cannot be formulated as a rigorous optimization problem. The reason is that semantic equivalence between attributes is inherently subjective and highly dependent on the context. As with any recommendation system, the user's ultimate preference may deviate from what the history infers. For instance, given the earlier example of "Medical center name" versus "Hospital name," it is logical for a method to infer from the existing schema that the user likely intends to merge them. Yet a user might insist on retaining both to preserve fine-grained distinctions, much like a consumer abruptly deviating from past behavior to purchase an entirely different product.

Given this inherent uncertainty, our Hybrid Integration Strategy is pragmatic rather than exhaustive: we defer to the diversity metric and LLM's judgment and perform moderate processing to consolidate obvious duplicates, without pursuing a perfect formulation or solution. Our experiments further support this pragmatic stance (see Section 6.3(2)), which shows that pruning the most conspicuous redundancies already yields substantial gains.

9.4 Generating Attribute Proposal Definitions

Since each attribute proposal varies in occurrence frequency, number of mentions, and mention length across source texts, directly gathered raw contexts can be highly inconsistent in length and style. This variability introduces bias to the LLM and could result in overly long input that hinders reliable reasoning. To address this, we first normalize each proposal's context into a concise definition using the LLM itself.

This step produces a standardized context(z) for every attribute proposal z that needs to be judged by the LLM, ensuring consistent style and length and improving fairness. Let $D^{u+}_{i_1}, D^{u+}_{i_2}, \ldots$ denote the pseudo-tables containing the attribute proposal z. The LLM instruction for generating a standard context is shown below.

Generating context(z) for attribute proposal z.

Your task is to summarize the context of an attribute and write a definition for it.
Provide the definition in this JSON structure:
{ "<name>": "<definition>", }

Explanations about the JSON structure:
- Output only the JSON data, where the key is the name of the attribute and the value is its definition.
- { dataset description }
- The name should be concise and in the same style with the examples.
- The name should be of proper granularity. Note that if it is too general, it cannot accurately describe the range of values; if it is too specific, it cannot cover all the values.
- The definition should precisely describe the meaning and characteristics of the attribute, enabling the reliable extraction of the attribute from other texts.
- The definition should be less than 50 words.
{ example definition }
Here is the context of the attribute to be defined:
In the text $W^u_{i_1}$, the z value are: { values in $D^{u+}_{i_1}$ }
In the text $W^u_{i_2}$, the z value are: { values in $D^{u+}_{i_2}$ }
...

Here, the "dataset description" the same as that in Extraction Intruction, and "example definitions" is the definition for known attributes in the schema, if there are any.

Table 5. Comparison of datasets. “Multiple” means whether each table contains multiple records.

Dataset	Input texts	Output table	Comments	Multiple
E2E [44, 45, 67]	human-written description	synthetically generated restaurant attributes	table description	No
RotoWire [64, 67]	human-written game summary	NBA box score	table description	Yes
WikiTableText [4, 67]	human-written table description	Wikipedia web tables	table description	No
WikiBio [28, 67]	filtered Wikipedia biographies	Wikipedia web tables	specialized documents	No
CPL [26]	Chinese private lending judgments	local tabular views of a KG	specialized documents	Yes
Incidents (Ours)	gun-violence news report [63]	manually-annotated tables about the victim, suspect and accident	naturally occurring text	Yes
Weather (Ours)	weather forecasts in news [63]	manually-annotated tables about weather details	naturally occurring text	Yes
Conversation	dialogue around services	aggregated final dialogue state	naturally occurring text (partially, because the service ontology is predefined)	No

Table 6. Dataset schema.

Dataset	**Entity**: Attributes
Incidents	**Accident**: Accident type, Accident date, Accident address, Number of rounds fired, Accident number, Personnel arrived time; **Victim**: Victim number, Victim status, Victim gender, Victim age, Victim based, Hospital name, Victim name, Victim race, Victim occupation, Victim vehicle; **Suspect**: Suspect gender, Suspect number, Suspect status, Suspect age, Suspect name, Suspect description, Suspect weapon, Suspect vehicle, Suspect occupation, Suspect based, Suspect race, Prison name
Weather	**Weather**: Temperature, Snow status, Cloud type, Sunset time, Rain status, Weather area, Time, Weather type, Wind speed, Weather compass direction, Maximum temperature, Weather frequency, Wind status, Minimum temperature, Sunrise time, Weather occurring chance, Wind direction, Location, Cloud status
Conversation	**Attraction**: Attraction name, Attraction area, Attraction type; **Hotel**: Hotel name, Hotel area, Hotel type, Hotel parking, Hotel price range, Hotel internet, Hotel stars, Hotel requested day, Hotel requested people, Hotel requested stay, Hotel confirmed name, Hotel confirmed reference; **Restaurant**: Restaurant name, Restaurant area, Restaurant food, Restaurant price range, Restaurant requested day, Restaurant requested people, Restaurant requested time, Restaurant confirmed name, Restaurant confirmed reference; **Taxi**: Taxi departure, Taxi destination, Taxi leave at, Taxi arrive by; **Train**: Train departure, Train destination, Train day, Train leave at, Train arrive by, Train requested people, Train confirmed reference

9.5 Discussion of existing datasets

Table 5 summarizes all open-source datasets used in previous table extraction from text methods in contrast to ours.

9.6 Data Annotation

We first deduplicate the original texts and remove corrupted texts. Three annotators (A, B, and C) participate in creating the table annotations. For each dataset, the annotation proceeds in three steps. First, the three annotators read the original CACAPO documentation and discussed the original attribute set to reach a consensus on the overall schema. Second, we sample 100 texts, which are independently annotated by A, B, and C. During annotation, each annotator is provided with the original attribute-value pairs and the complete table extraction schema. They are asked to fill in missing values, correct errors in the original annotations, and organize the correct values into multiple records (rows) to form a complete table. Disagreements are discussed until consensus is reached. The remaining texts are then split into two parts and assigned to B and C, respectively. From each part, we sample another 100 texts for A to annotate independently. The pairwise agreement between A and B is 88%, and between A and C is 82%.

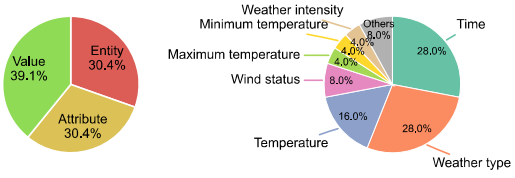


Fig. 13. Disagreement types on Weather.

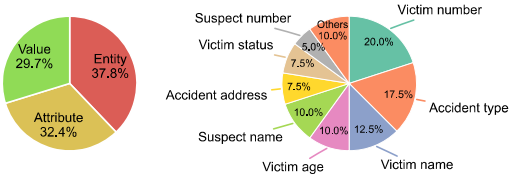


Fig. 14. Disagreement types on Incidents.

We categorize observed pairwise disagreements into three hierarchical levels, from coarse to fine-grained: (1) **Entity-level**: Disagreements on the presence of entities, corresponding to how many rows a table should contain. (2) **Attribute-level**: Disagreements on which attributes are instantiated for a given entity, without any Entity-level disagreement. This corresponds to determining the non-empty columns within a table row. (3) **Value-level**: Disagreements on the value of a specific attribute, without any Entity- or Attribute-level disagreement. This corresponds to the cell content. We analyze the distribution of disagreement types and the attribute-level breakdown of where disagreements occur, visualized in Figure 14 and 13.

9.7 Variability Comparison

We also conduct a statistical examination of the **variability** of our new benchmark datasets with that of Conversation. We study the variability in two aspects: how an attribute is manifest in the text (Challenge 1) and which attributes are mentioned in a text (Challenge 2).

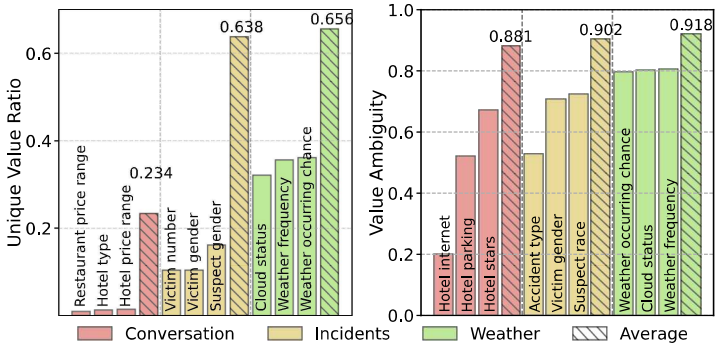


Fig. 15. Attribute-level variability comparison.

To underscore Challenge 1, we compare the attribute-level variability: (1) Unique Value Ratio, $\in (0, 1]$: #unique values / #values. A higher score means the attribute manifests more unique values among all its mentions in the text. (2) Value Ambiguity $\in [0, 1]$: the normalized entropy of value distribution. A higher score indicates that no single value dominates the corpus, and the attribute lacks a global default interpretation. Figure 15 shows the average and the lowest scores (the "easiest") for each dataset. The former demonstrates the general difficulty while the latter reveals the lower-bound difficulty. These statistics imply the difficulty order of the datasets: Weather > Incidents > Conversation, aligning with the observation in Table 2.

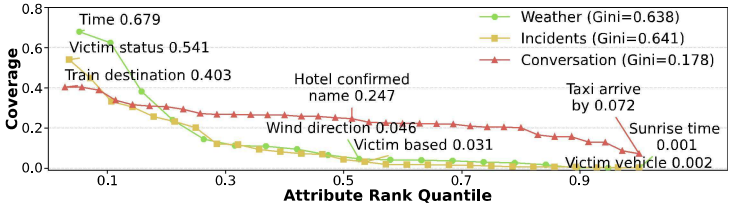


Fig. 16. Attribute Coverage ranked curve.

For Challenge 2, we depict the Attribute Coverage (i.e. the ratio of texts mentioning the attribute) in Figure 16. Apparently, the long-tail phenomenon of our new benchmarks is more significant

with higher Gini than the Conversation dataset, reflecting the fact that Incidents and Weather contain more rare attributes.

9.8 Discussion on Structured-F1

We discuss the computational complexity of structured-F1. Assume D has n rows and m columns while $\hat{D}$ has $\hat{n}$ rows and $\hat{m}$ columns. Let c denote the amortized cost to calculate the similarity between two cell contents. To compute DF1 for some record pair, it requires $O(m\hat{m}c)$ to get the similarity matrix between header names, as in Figure 3(1), and $O((m+\hat{m})c)$ to average the similarity of matched values. We need to compute DF1 for all $n\hat{n}$ record pairs, the f_k similarity matrix could be reused, so the overall complexity for structured-F1 is $C1 = O(m\hat{m}c + n\hat{n}(m+\hat{m})c)$.

In contrast, if the table is flattened, there are mn and $\hat{m}\hat{n}$ triples. So, the complexity of comparing triples is $C2 = O(mn\hat{m}\hat{n}c)$.

For a reasonable system, the size of D resembles that of $\hat{D}$, thus $m \approx \hat{m}$ and $n \approx \hat{n}$, so $C1 = O(m^2c + 2n^2mc)$, $C2 = O(m^2n^2c)$. When $n^2 > 1$ and $m > 3$, $C1 \leq C2$ and structured-F1 is more efficient than triple F1; in other cases, $C1 \approx C2$. In practice, the m and n are usually small integers, so it is fair to say our metric does not increase the evaluation complexity.

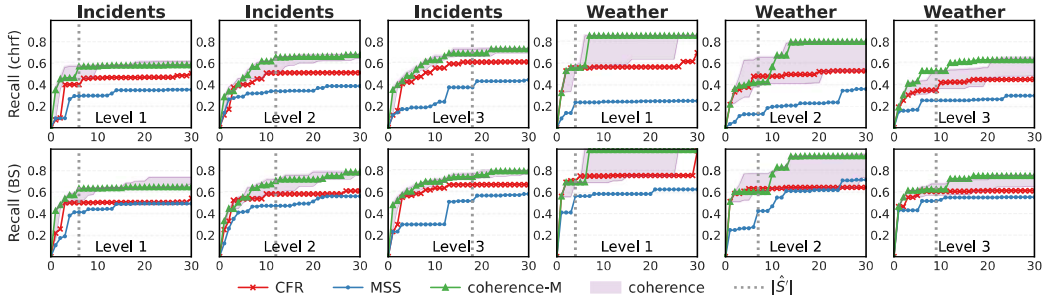


Fig. 17. Recommendation recall with Llama-3-8B.

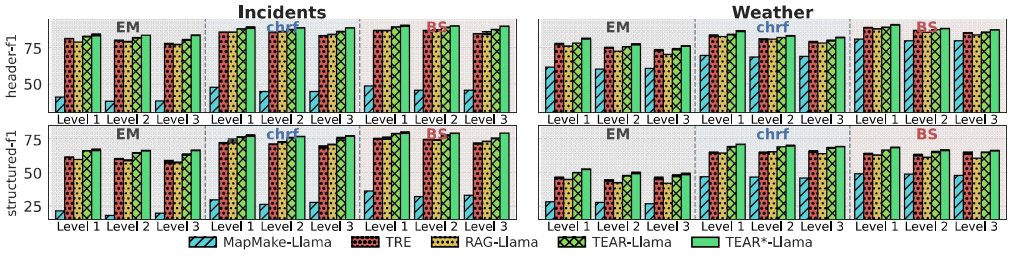


Fig. 18. Table extraction with text-driven attributes, Llama backbone.

9.9 Other Results

Figure 17 and Figure 18 are the results with Llama-3-8B.